

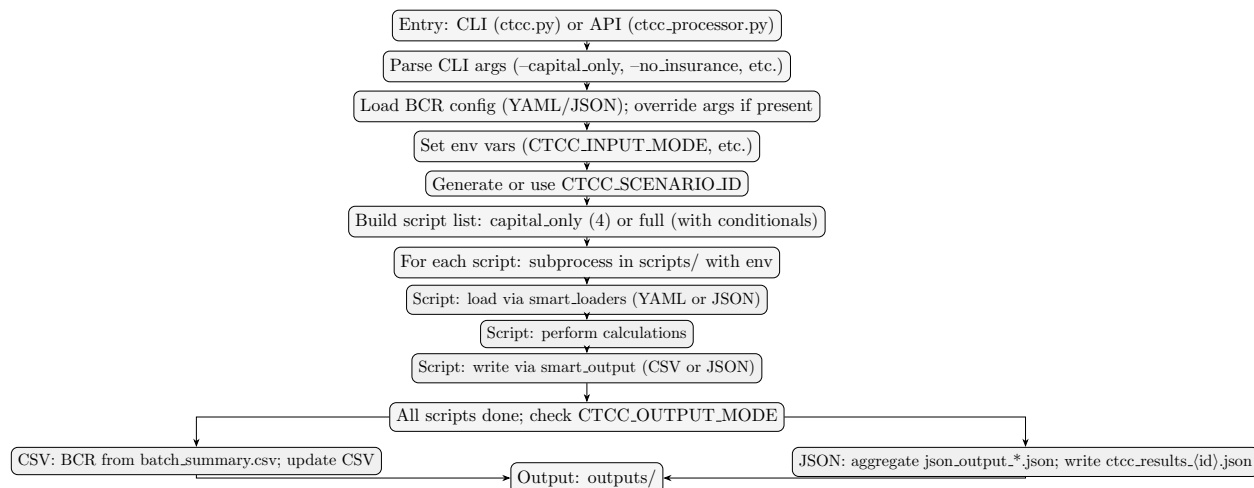
Chapter 1

Master Code Map

1.1 Introduction

This chapter maps every step of CTCC's execution: from entry (CLI or API) through argument parsing, environment setup, script list construction, sequential run of calculation scripts, and final BCR and output (CSV or JSON). A full workflow diagram in Mermaid syntax is available in `docs/ctcc_execution_workflow.txt`.

1.2 Execution Flow Diagram



1.3 Every Step (Numbered)

1. Entry: user runs `python ctcc.py` (CLI) or POSTs to API `/api/ctcc/calculate` (server writes combined JSON to temp file and calls `ctcc.py` via subprocess).
2. Parse command-line arguments: `--norisk`, `--simple`, `--no_wf_liability`, `--no_emissions`, `--no_linelosses`, `--capital_only`, `--no_oandm`, `--no_insurance`, `--no_delay_costs`,

- `--no_wildfire`, `--no_outages`, `--no_congestion`, `--no_curtailment`, `-j`, `-o`, `--id`, etc.
3. Load primary BCR config from YAML or JSON (based on `CTCC_INPUT_MODE`); if present, merge with CLI flags (config overrides).
 4. Build `BCRConfig` from args and set environment variables (`CTCC_NO_CONGESTION`, `CTCC_NO_WF_LIABILITY`, etc.).
 5. Generate `CTCC_SCENARIO_ID` if not set (e.g. `YYYYMMDD_HHMMSS_microseconds`), or use existing (e.g. from `--id` or API).
 6. If JSON input mode: set `CTCC_JSON_DATA_FILE` (and `os.environ`) to the combined data file path.
 7. Build script list: if `--capital_only`, list is `weighted_miles.py`, `build_costs.py`, `row_costs.py`, `environmental_mitigation.py`. Otherwise, start with `weighted_miles.py`, `build_costs.py`, `row_costs.py`, `environmental_mitigation.py`, `revenue.py`; then append `insurance_costs.py` unless `--no_insurance`, `delay_costs.py` unless `--no_delay_costs`, `wildfire_costs.py` unless `--no_wildfire`, `outage_costs.py` unless `--no_outages`; then `congestion_curtailment_reduction.py` unless `--no_congestion_curtailment_reduction`, `energy_losses.py`; then `oandm.py` unless `--no_oandm`; then `emissions.py` unless `--no_emissions`, `line_loss_costs.py` unless `--no_line_losses`.
 8. For each script in order: run `sys.executable script_name` as subprocess with `cwd=scripts`, `env=os.environ.copy()`; capture stdout/stderr; track success/failure.
 9. Each script: (a) loads data via `smart_loaders` (dispatches to YAML or JSON loaders per `CTCC_INPUT_MODE`); (b) performs its calculations; (c) writes results via `smart_output` (CSV or JSON per `CTCC_OUTPUT_MODE`).
 10. After all scripts: if no successful runs, exit with error. Otherwise, if `CTCC_OUTPUT_MODE=csv`: run `calculate_and_display_bcr` on `batch_summary.csv`, update `batch_summary.csv` with BCR metrics via `CSVOutputManager`. If `CTCC_OUTPUT_MODE=json`: `aggregate_json_outputs` loads and merges `json_output.<id>*.json`, `build_bcr_data_from_json` extracts flat dict, BCR computed via `calculate_benefits/calculate_costs/calculate_bcr_metrics`, `write_final_json_output` writes `ctcc_results-<id>.json`, then intermediate JSON files are removed.
 11. Exit: code 0 if all scripts succeeded; 1 if any failed.

1.4 Script Inclusion Table

Table 1.1 shows when each script is included in the run list and its purpose in one line.

With `--capital_only`, only the first four scripts (`weighted_miles`, `build_costs`, `row_costs`, `environmental_mitigation`) are run.

Script	When included	Purpose
weighted_miles.py	Always	Terrain-adjusted miles; prerequisite for build_costs, insurance
build_costs.py	Always	Construction costs (conductor, structure, converter; AFUDC/PV)
row_costs.py	Always	Right-of-way costs
environmental_mitigation.py	Always	Environmental mitigation costs
revenue.py	Always (full run)	Revenue requirement
insurance_costs.py	If not <code>--no_insurance</code>	Insurance costs
delay_costs.py	If not <code>--no_delay_costs</code>	Construction/delay costs
wildfire_costs.py	If not <code>--no_wildfire</code>	Wildfire risk costs
outage_costs.py	If not <code>--no_outages</code>	Outage risk costs
congestion_curtailment_reduction.py	Always (full run)	Congestion and curtailment benefits
energy_losses.py	Always (full run)	Loss data; prerequisite for line_loss_costs
oandm.py	If not <code>--no_oandm</code>	O&M costs
emissions.py	If not <code>--no_emissions</code>	Emissions costs
line_loss_costs.py	If not <code>--no_linelosses</code>	Line loss costs/benefits

Table 1.1: Scripts: inclusion condition and one-line purpose.